Solution: Shortest Path in Binary Matrix

Problem Statement

Given a matrix of size `m x n`, return the `shortest path` from the top-left corner `(0,0)` to the bottom-right corner `(N-1, N-1)`. If there is no such path, the output should be `-1`.

Follow the rules below while calculating the shortest path.

- You can move horizontally, vertically, and diagonally to adjacent cells.
- The visited cell should be `0`.
- The path length is counted as the number of steps taken.

Examples

1. **Example 1**:

- **Input**:

```
[[0, 1, 0],
 [0, 0, 0],
 [0, 1, 0]]
```

- **Expected Output**: `3`
- **Justification**: The shortest path is (0, 0) -> (1, 1) -> (2, 2), totaling 3 steps.

2. **Example 2**:

- **Input**:

```
[1, 1, 1],
 [0, 1, 1],
 [0, 0, 0]]
```

- **Expected Output**: `-1`
- **Justification**: The top-left cell can't be visited, as its value is 1. So, it returns -1.

3. **Example 3**:

- **Input**:

```
[[0, 0, 0],
 [0, 1, 0],
 [0, 0, 0]]
```

- **Expected Output**: `4`
- **Justification**: The shortest path is (0, 0) -> (1, 0) -> (2, 1) -> (2, 2), totaling 4 steps.

Solution

To solve this problem, we'll use the Breadth-First Search (BFS) algorithm. BFS is suitable for finding the shortest path in a graph, and here, our matrix can be viewed as a graph where each cell is a node connected to its 8 adjacent cells.

BFS explores the neighbors of a node before moving to the next level, ensuring that the first time we reach the destination, it's via the shortest path. The algorithm will be implemented using a queue to keep track of cells to visit and their distance from the start. We'll also use a way to mark visited cells to avoid revisiting them. This approach is effective because it systematically explores paths from the start and guarantees the shortest path due to the nature of BFS.

Step-by-step algorithm

1. **Initialization**:
    - Check if the start (0,0) or end (N-1, N-1) cells are blocked. If so, return -1.
    - Initialize the `queue` with the starting cell (0,0) and its path length (1).
    - Set up the `visited` matrix, marking the start cell as visited.
    - Define `directions` for the 8 possible moves.
2. **BFS Loop**:
    - While the `queue` is not empty:
        - Dequeue the first element (current cell and its path length).
        - Check if the current cell is the destination (N-1, N-1). If so, return its path length.
        - For each direction in `directions`:
            - Calculate the coordinates of the adjacent cell.
            - Check if the move is valid (cell is within bounds, not blocked, and not visited).
            - If valid, mark the cell as visited, and enqueue it with an incremented path length.
3. **No Path Found**:
    - If the BFS loop completes without finding the destination, return -1.

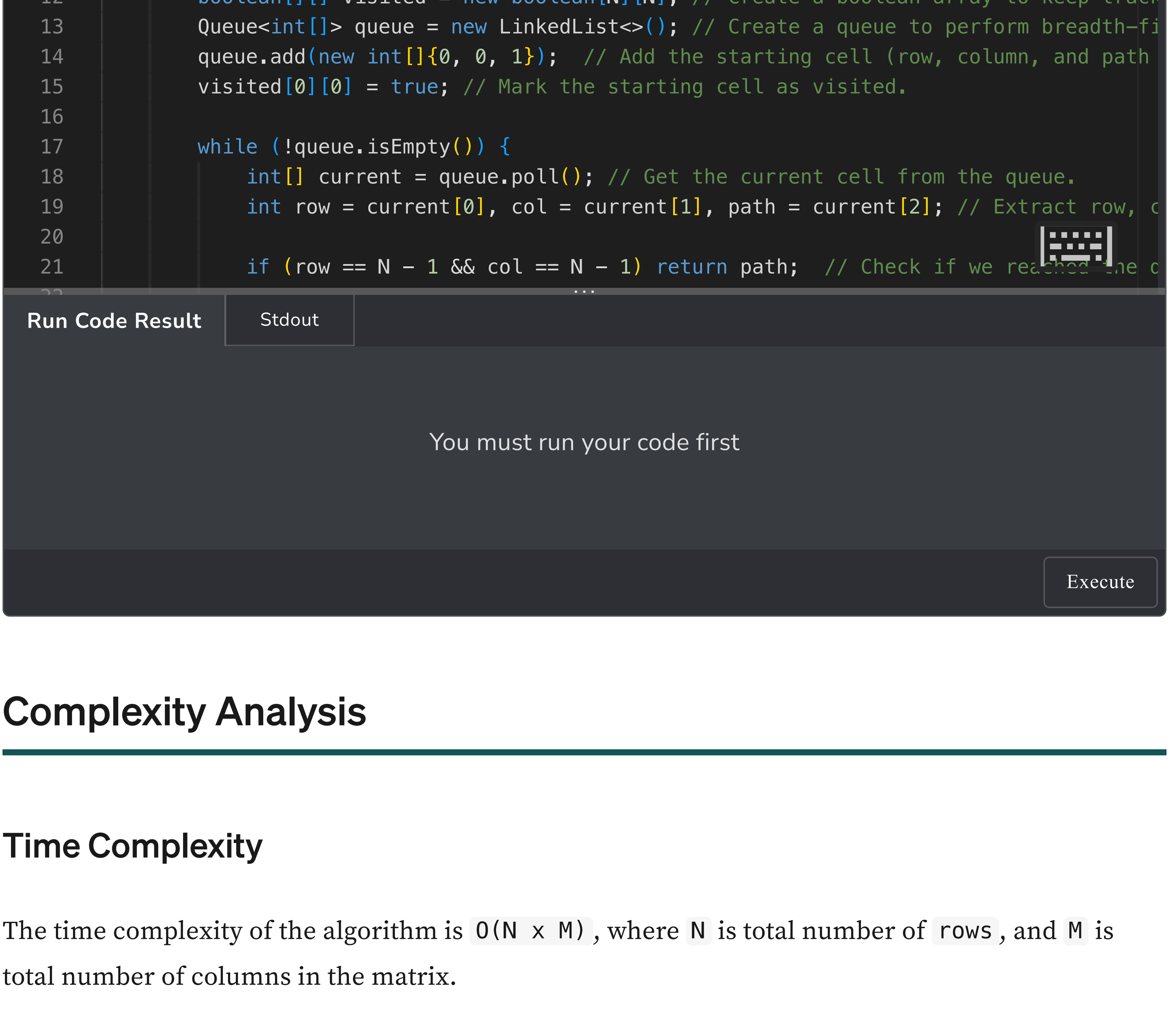
Algorithm Walkthrough

Let's consider the Input:

```
[[0, 0, 0],
 [0, 1, 0],
 [0, 0, 0]]
```

- **Initialization**: Start from (0,0) with path length 1. `queue = [(0, 0, 1)]`, where (0, 0) represents the (row, col) of the matrix, and `1` represents the shortest path length till the current cell. `visited[0][0] = true`.
- **Step 1**: Pop (0, 0, 1). Check all 8 directions:
    - Right (0,1): Valid and unvisited. Enqueue (0,1,2).
    - Down (1,0): Valid and unvisited. Enqueue (1,0,2).
    - Down-right (1,1): Blocked.
    - Update `queue` to `[(0,1,2), (1,0,2)]`.
- **Step 2**: Pop (0, 1, 2). Check all 8 directions:
    - Right (0,2): Valid and unvisited. Enqueue (0,2,3).
    - Down-right (1, 2): Valid and unvisited. Enqueue (1,2,3)
    - Update `queue` to `[(1,0,2), (0,2,3), (1, 2, 3)]`.
- **Step 3**: Pop (1, 0, 2). Check all 8 directions:
    - Down (2,0): Valid and unvisited. Enqueue (2,0,3).
    - Down-right (2,1): Valid and unvisited. Enqueue (2,1,3).
    - Update `queue` to `[(0,2,3), (1, 2, 3), (2,0,3), (2, 1, 3)]`.
- **Step 4**: Pop (0, 2, 3). Check all 8 directions:
    - All adjacent nodes are visited.
    - Update `queue` to `[(1, 2, 3), (2,0,3), (2, 1, 3)]`.
- **Step 5**: Pop (1, 2, 3). Check all 8 directions:
    - Down (2,2): Valid and unvisited. Enqueue (2,2,4).
    - Down-left (2, 1): Valid and unvisited. Enqueue (2,1,4)
    - Update `queue` to `[(2,0,3), (2, 1, 3), (2, 2, 4), (2, 1, 4)]`.
- **Step 6**: Pop (2, 0, 3). Check all 8 directions:
    - All adjacent nodes are visited.
    - Update `queue` to `[(2, 1, 3), (2, 2, 4), (2, 1, 4)]`.
- **Step 7**: Pop (2, 1, 3). Check all 8 directions:
    - All adjacent nodes are visited.
- **Step 8**: Pop (2, 2, 4). This is the destination cell.
- Return the path length 4 as it's the shortest path to the destination.

Code



```java
import java.util.LinkedList;
import java.util.Queue;

public class Solution {
    // Define eight possible directions to move: up, down, left, right, and diagonally
    private static final int[][] DIRECTIONS = new int[][]{{1, 0}, {0, 1}, {1, 1}, {-1, 0

    public int shortestPathBinaryMatrix(int[][] grid) {
        int N = grid.length; // Get the size of the grid (assuming it's a square grid).
        if (grid[0][0] == 1 || grid[N - 1][N - 1] == 1) return -1;  // Check if start or

        boolean[][] visited = new boolean[N][N]; // Create a boolean array to keep track
        Queue<int[]> queue = new LinkedList<>(); // Create a queue to perform breadth-fi
        queue.add(new int[]{0, 0, 1});  // Add the starting cell (row, column, and path
        visited[0][0] = true; // Mark the starting cell as visited.

        while (!queue.isEmpty()) {
            int[] current = queue.poll(); // Get the current cell from the queue.
            int row = current[0], col = current[1], path = current[2]; // Extract row, c

            if (row == N - 1 && col == N - 1) return path;  // Check if we reached the c
```

You must run your code first

Complexity Analysis

Time Complexity

The time complexity of the algorithm is `O(N x M)`, where `N` is total number of `rows`, and `M` is total number of columns in the matrix.

The code uses a breadth-first search (BFS) approach to find the shortest path in the binary matrix. In the worst case, the BFS will visit each cell once. Therefore, the time complexity is `O(N x M)`, where `N x M` is the size of the grid.

Space Complexity

The space complexity of the algorithm is `O(N x M)`.

The `visited` array is of size `N x M,` which requires `O(N x M)` space to store the visited status of each cell. The queue used for BFS can have at most `N x M` cells in it at any point. Therefore, the space complexity is also `O(N x M)`.